

Hooks Avançados e Gerenciamento de Estado Global

Sumário:

1. Hooks Avançados.....	2
1.1 Hook useReducer.....	2
1.2 Hook useMemo.....	7
1.3 Hook useCallback.....	12
1.4 React.memo.....	14
1.5 Hooks Customizados.....	16
2. Compartilhamento de Estado Global.....	19
2.1 Principais tipos de estado em React.....	19
2.2 Context API.....	21
3. Boas Práticas no Uso de Hooks em React.....	29
3.1 Organizar a Lógica de Hooks na Ordem Correta.....	29
3.2 Evitar useEffect sem Necessidade.....	29
3.3 Usar Corretamente as Dependências no useEffect.....	30
3.4 Dividir Efeitos Colaterais em Diferentes useEffect.....	30
3.5 Uso de useMemo e useCallback para Otimizações.....	31
3.6 Encapsular Lógica Reutilizável em Hooks Customizados.....	31
Referências.....	33

1. Hooks Avançados

Os hooks foram introduzidos no React 16.8 para permitir o uso de estado e outros recursos de React em **componentes funcionais**. Eles são funções que permitem acessar funcionalidades do React, como gerenciamento de estado e efeitos colaterais, sem a necessidade de usar componentes de classe.

Além dos hooks básicos como **useState** e **useEffect**, o React oferece hooks mais avançados que permitem resolver problemas mais complexos ou melhorar a performance e reutilização de código. A seguir, vamos explorar alguns desses hooks avançados do React.

1.1 Hook **useReducer**

O **useReducer** é um Hook do React usado para gerenciar **estados mais complexos** ou quando há **várias ações** que precisam **modificar o estado** de maneiras diferentes. Ele é uma alternativa ao **useState**, fornecendo uma maneira mais estruturada de atualizar o estado, especialmente quando ele envolve múltiplas propriedades ou regras de atualização complexas.

Veja abaixo a sintaxe básica do **useReducer**:

```
const [estado, dispatch] = useReducer(reducer, estadoInicial);
```

Assim, ele é uma função que toma dois parâmetros principais:

- **reducer**: Uma função que recebe o estado atual e uma ação, e retorna o novo estado.
- **estadoInicial**: O valor inicial do estado.

Veja que a função **useReducer** retorna um array com dois itens:

- **estado**: O estado atual do componente.
- **dispatch**: Uma função usada para disparar ações que modificarão o estado.

O **useReducer** segue o **conceito de redução**, ou seja, ele recebe o estado atual e uma ação, processa essa ação e retorna o novo estado. A lógica de como o estado será atualizado é centralizada dentro da função **reducer**.

Veja no quadro abaixo um exemplo simples onde criamos um contador que pode ser incrementado ou decrementado com base em um conjunto de ações:

```
import { useReducer } from "react";
// Função reducer para gerenciar o estado
function reducer(state, action) {
  switch (action.type) {
    case "incrementar":
      return { contador: state.contador + 1 };
    case "decrementar":
      return { contador: state.contador - 1 };
    default:
      return state;
  }
}

function Contador() {
  // useReducer com o estado inicial do contador
  const [state, dispatch] = useReducer(reducer, { contador: 0 });

  return (
    <div>
      <p>Contador: {state.contador}</p>
      <button onClick={() => dispatch({ type: "incrementar" })}>
        Incrementar
      </button>
      <button onClick={() => dispatch({ type: "decrementar" })}>
        Decrementar
      </button>
    </div>
  );
}

export default Contador;
```

Neste exemplo:

- **A função reducer:**
 - Recebe dois parâmetros: o estado atual (**state**) e a ação (**action**).
 - Com base no tipo da ação (**incrementar** ou **decrementar**), ela retorna o novo estado do **contador**.

- **useReducer(reducer, { contador: 0 }):**
 - Inicializa o estado com { **contador: 0** } e define a função **dispatch** que será usada para disparar ações.
- **dispatch({ type: 'incrementar' }):**
 - Quando o botão "Incrementar" é clicado, a função **dispatch** dispara a ação com o tipo 'incrementar', que faz o **reducer** retornar um novo estado com contador incrementado em **1**.

Veja abaixo o resultado no navegador quando clicamos 1 vez no botão “Incrementar”:

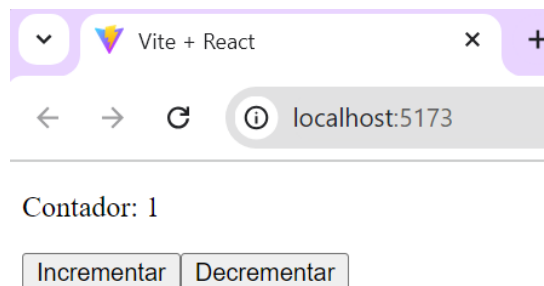


Figura 1: Exemplo de um funcionamento de estado com useReducer

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". O conteúdo da página exibe um contador com o valor "1" e dois botões abaixo: "Incrementar" e "Decrementar", que provavelmente permitem aumentar ou diminuir o valor do contador.

1.1.1 Gerenciamento de Estado Complexo e Lógica de Atualização

Vimos que o **useReducer** é muito útil para estados complexos, como quando você precisa gerenciar objetos com várias propriedades ou realizar atualizações condicionais baseadas em diferentes tipos de ações. Neste quadro abaixo temos um exemplo onde vamos gerenciar o perfil de um usuário, que contém várias propriedades (nome, idade e cidade), e permitiremos atualizar cada uma delas com diferentes ações.

```
import { useReducer } from "react";
// Função reducer para gerenciar o estado do perfil
function perfilReducer(state, action) {
  switch (action.type) {
    case "atualizarNome":
      return {
        ...state,
        nome: action.payload,
      };
  }
}
```

```
    case "atualizarIdade":
      return {
        ...state,
        idade: action.payload,
      };

    case "atualizarCidade":
      return {
        ...state,
        cidade: action.payload,
      };
    default:
      return state;
  }
}

function PerfilUsuario() {
  // Estado inicial do perfil
  const [perfil, dispatch] = useReducer(perfilReducer, {
    nome: "João",
    idade: 25,
    cidade: "São Paulo",
  });
  return (
    <div>
      <h2>Perfil do Usuário</h2>
      <p>Nome: {perfil.nome}</p>
      <p>Idade: {perfil.idade}</p>
      <p>Cidade: {perfil.cidade}</p>

      { /* Botões para disparar ações que atualizam o estado */ }
      <button onClick={() => dispatch({ type: "atualizarNome", payload: "Maria" })}>
        Mudar Nome para Maria
      </button>
      <button onClick={() => dispatch({ type: "atualizarIdade", payload: 30 })}>
        Mudar Idade para 30
      </button>
      <button onClick={() => dispatch({ type: "atualizarCidade", payload: "Rio de
Janeiro" })}>
        Mudar Cidade para Rio de Janeiro
      </button>
    </div>
  );
}

export default PerfilUsuario;
```

Neste exemplo:

- Função **perfilReducer**:
 - Essa função recebe o estado atual (**state**) e uma ação (**action**).
 - Dependendo do **type** da ação (por exemplo, **'atualizarNome'**, **'atualizarIdade'**, ou **'atualizarCidade'**), o **reducer** retorna um **novo estado** com a propriedade correspondente atualizada.
 - O **payload** da ação contém o novo valor que será usado para atualizar a propriedade relevante.
- Estado Inicial:
 - O estado inicial é um objeto com três propriedades: nome, idade e cidade.
 - **useReducer(perfilReducer, {nome: 'João', idade: 25, cidade: 'São Paulo' })** inicializa o estado do perfil com esses valores.
- Disparo das Ações:
 - Cada botão dispara uma ação diferente, que é passada para o **dispatch**.
 - O **dispatch** envia a ação para o **reducer**, que atualiza o estado com base no tipo de ação. Por exemplo, ao clicar no botão "Mudar Nome para Maria", a ação { **type: 'atualizarNome', payload: 'Maria' }** é enviada, e o nome no estado é atualizado para "Maria"
- Renderização:
 - O componente renderiza automaticamente quando o estado do perfil é atualizado, refletindo as mudanças no **nome**, **idade** ou **cidade**.

Veja abaixo o resultado no navegador quando clicamos 1 vez no botão “Incrementar”:

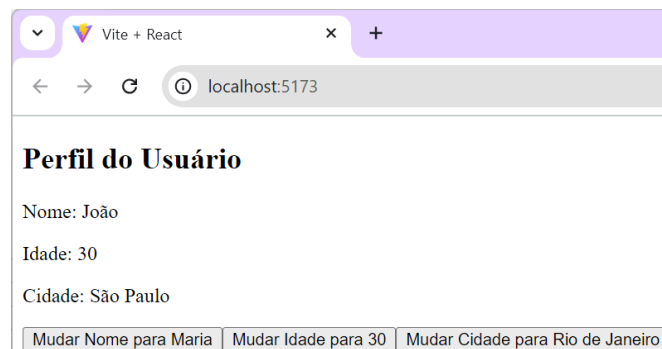


Figura 2: Exemplo de um funcionamento de estado com useReducer com estado complexo

Descrição: A imagem mostra uma página web. O conteúdo da página exibe um "Perfil do Usuário" com as informações: "Nome: João", "Idade: 30" e "Cidade: São Paulo". Abaixo dessas informações, há três botões: "Mudar Nome para Maria", "Mudar Idade para 30", e "Mudar Cidade para Rio de Janeiro".

1.1.2 Comparação useState vs useReducer

Veja na tabela abaixo uma comparação entre os Hooks **useState** e **useReducer**:

Critério	useState	useReducer
Complexidade do Estado	Melhor para estados simples	Ideal para estados complexos ou lógicos
Lógica de Atualização	Simples, com chamadas diretas de setState	Centraliza a lógica na função reducer
Número de Estados	Usa múltiplos useState para gerenciar várias variáveis	Usa um único estado complexo
Vantagens	Fácil de implementar, direto	Estrutura a lógica de atualização de estado
Desvantagens	Pode ficar complicado com muitos estados	Mais verboso, mas mais controlado

1.2 Hook useMemo

O **useMemo** é um hook que **memoriza o resultado de uma função** e só recalcula esse resultado quando as dependências da função mudam. Ele é útil quando você tem **cálculos pesados** ou **operações custosas** (processos que consomem muito tempo ou recursos computacionais) que não precisam ser executadas em cada renderização.

Veja abaixo sua sintaxe:

```
const valorMemorizado = useMemo(() => {  
  // Função que realiza o cálculo pesado  
  return resultado;  
}, [dependencias]);
```

Onde:

- **valorMemorizado**: O valor calculado que será memorizado.
- **useMemo()**: Recebe dois argumentos:
 - Uma **função de callback** que realiza o cálculo ou a operação que você deseja memorizar.
 - Um **array de dependências**. O React só recalculará o valor se alguma dessas dependências mudar.
- **dependencias**: Variáveis que, quando alteradas, forçarão o recálculo do valor. Se essas variáveis não mudarem, o valor memorizado será reutilizado.

1.2.1 Por que usar useMemo?

A memorização é útil para **cálculos computacionalmente custosos** ou para **evitar renderizações desnecessárias** de componentes. Por padrão, no React, cada vez que o componente é re-renderizado, todas as funções dentro desse componente são recriadas. Isso geralmente não é um problema, mas em alguns casos pode causar problemas de performance ou comportamentos indesejados, como quando você passa uma função para um componente filho como **prop**, esse componente filho pode ser re-renderizado sempre que a função for recriada, mesmo que seu comportamento ou valor não tenha mudado.

1.2.2 Exemplo simples de uso do useMemo

Vamos criar um exemplo simples para mostrar como **useMemo** pode ser útil em evitar o recálculo desnecessário de uma função em cada renderização.

A) Sem useMemo

Neste exemplo, um componente que **filtra uma lista de produtos** com base na busca do usuário. Sem o uso de **useMemo**, a função de **filtragem será recalculada** em toda renderização, mesmo quando não for necessário.

```
import { useState } from "react";

// Lista de produtos
const produtos = ["Notebook", "Celular", "Tablet", "Teclado", "Mouse", "Monitor", "Impressora"];

// Componente que exibe a lista filtrada de produtos
function App() {
  const [busca, setBusca] = useState("");

  // Função de filtragem é recalculada em toda renderização
  const produtosFiltrados = produtos.filter((produto) => {
    console.log("Filtrando produtos...");
    return produto.toLowerCase().includes(busca.toLowerCase());
  });
}
```



```
return (  
  <div>  
    <input type="text" placeholder="Buscar produtos" value={busca}  
    onChange={(e) => setBusca(e.target.value)} />  
    <ul>  
      {produtosFiltrados.map((produto, index) => (  
        <li key={index}>{produto}</li>  
      ))}  
    </ul>  
  </div>  
);  
}  
export default App;
```

Neste exemplo:

- Cada vez que o usuário digita algo no campo de busca, o componente **App** é renderizado novamente.
- Toda vez que o componente renderiza novamente, a função de filtragem é recalculada, mesmo que a lista de produtos ou a busca não tenha mudado.
- Se a lista de produtos fosse muito grande, isso poderia causar novos **cálculos desnecessários** e impacto na performance.

Veja na imagem a seguir o resultado da execução deste código no navegador:

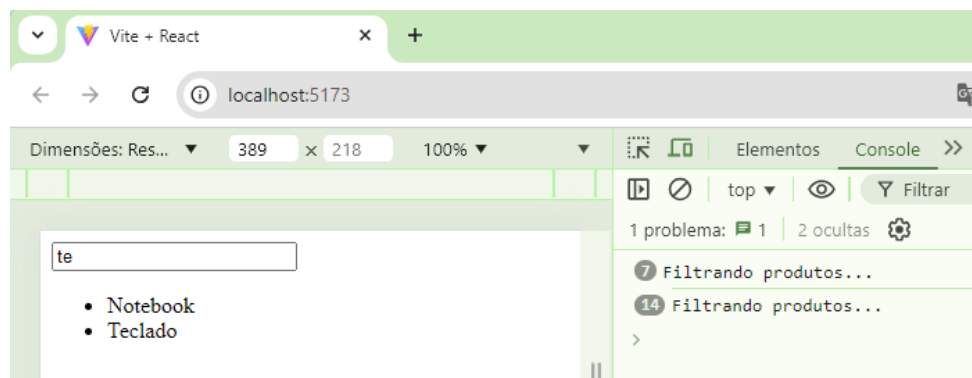


Figura 3: Exemplo de cálculo custoso sem uso do Hook useMemo

Descrição: A imagem mostra uma página web local com uma barra de pesquisa onde "te" foi digitado, filtrando os itens "Notebook" e "Teclado". o lado direito, mostra a aba "Console" do DevTools com a mensagem "Filtrando produtos..." repetida 21 vezes.

Neste exemplo, cada vez que o campo de busca é alterado, o filtro é executado novamente, mesmo que o cálculo do filtro seja o mesmo.

OBS: Caso você observe no seu console que a mensagem "Filtrando produtos..." está aparecendo mais do que 14 vezes, isso pode estar sendo influenciado pelo comportamento do `<React.StrictMode>` (geralmente usado no arquivo `main.jsx`). Esta é uma ferramenta do React que executa verificações adicionais durante o desenvolvimento, como a renderização dupla.

B) Com useMemo

Agora, vamos usar `useMemo` para **memorizar o resultado da filtragem**. Isso garantirá que a lista de produtos filtrados só seja recalculada quando o valor do campo de busca mudar.

```
import React, { useState, useMemo } from "react";
// Lista de produtos
const produtos = ["Notebook", "Celular", "Tablet", "Teclado", "Mouse",
"Monitor", "Impressora"];
function App() {
  const [busca, setBusca] = useState("");
  // Memoriza o resultado do filtro e só recalcula quando 'busca' mudar
  const produtosFiltrados = useMemo(() => {
    console.log("Filtrando produtos...");
    return produtos.filter((produto) =>
      produto.toLowerCase().includes(busca.toLowerCase())
    );
  }, [busca]);
  return (
    <div>
      <input type="text" placeholder="Buscar produtos" value={busca}
onChange={(e) => setBusca(e.target.value)} />
      <ul>
        {produtosFiltrados.map((produto, index) => (
          <li key={index}>{produto}</li>
        ))}
      </ul>
    </div>
  );
}
export default App;
```

Neste exemplo:

- Usamos o **useMemo** para memorizar o resultado da filtragem de produtos.
- O filtro só é recalculado se o resultado de **busca** mudar. Se essas dependências não mudarem, o React usa o valor memorizado e não executa novamente a função de filtragem.
- Isso evita **novos cálculos desnecessários** da filtragem de produtos, especialmente em renderizações onde o valor de busca não mudou.

Veja na imagem a seguir o resultado da execução deste código no navegador, observando as mensagens no console do Google DevTools, quando clicamos cinco vezes no botão “Incrementar Contador” e uma vez no botão “Aumentar Valor”:

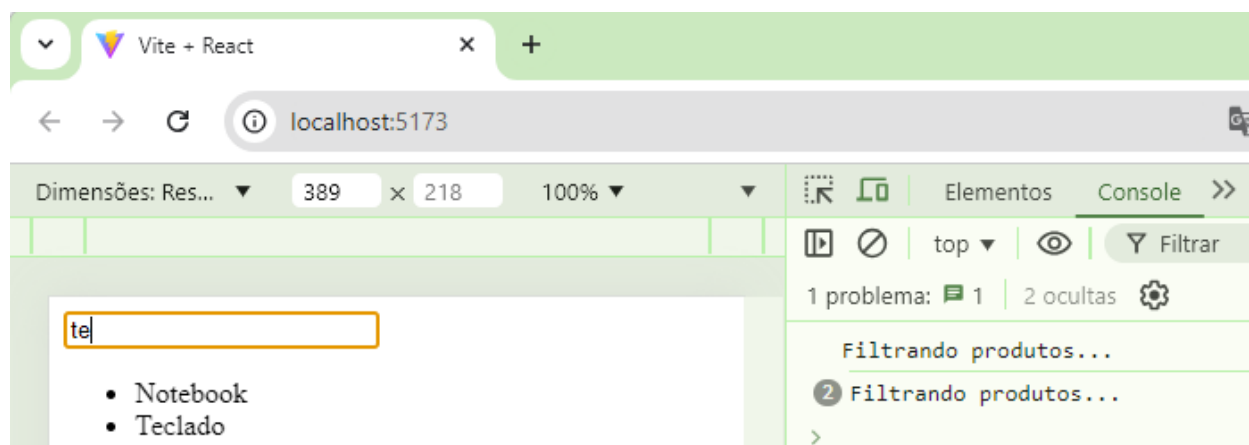


Figura 4: Exemplo de cálculo custoso com uso do Hook `useMemo`

Descrição: A imagem mostra uma página web local com uma barra de pesquisa onde "te" foi digitado, filtrando os itens "Notebook" e "Teclado". O lado direito, mostra a aba "Console" do DevTools com a mensagem "Filtrando produtos..." repetida 3 vezes.

Agora, o cálculo de filtragem só é feito quando o valor da busca ou a lista de produtos muda, evitando novos cálculos desnecessários.

1.2.3 Quando Usar `useMemo`?

Ele deve ser usado em situações específicas, como:

- **Cálculos Complexos ou Operações Custosas:** Quando você tem uma operação que envolve cálculos complexos ou que processa uma grande quantidade de

dados, usar **useMemo** pode evitar que esses cálculos sejam repetidos em toda renderização do componente.

- **Evitar Recalculos em Renderizações Frequentes:** Se você tem funções que dependem do estado ou de props e precisam realizar cálculos com base nessas informações, o **useMemo** evita que esses cálculos sejam refeitos toda vez que o componente renderiza.
- **Filtragem ou Transformação de Grandes Arrays:** Se você está filtrando ou transformando grandes arrays de dados, como uma lista de produtos, usuários ou itens, a filtragem pode se tornar custosa se for feita repetidamente em toda renderização. Nesse caso, **useMemo** evita recalcular a filtragem a menos que o array ou o critério de filtragem mude.

1.2.4 Quando Não Usar useMemo?

Evite usar **useMemo** nas seguintes situações:

- **Cálculos triviais:** Se o cálculo ou operação é simples e não afeta a performance, usar **useMemo** pode ser desnecessário e apenas adicionar complexidade.
- **Otimização Prematura:** Não use **useMemo** apenas por hábito ou com o objetivo de otimização prematura. Ele deve ser usado somente quando você identificar um problema de performance real.
- **Componentes que Renderizam Infreqüentemente:** Se o componente raramente é re-renderizado, o uso de **useMemo** pode ser desnecessário, pois o ganho de performance não será significativo

1.3 Hook useCallback

O **useCallback** é um hook que **memoriza uma função**. Ele garante que a função tenha a **mesma referência** entre renderizações, a menos que uma ou mais de suas dependências mudem. Isso evita que a função seja **recriada** em cada renderização do componente, o que pode ser importante para otimizar callbacks usados em eventos (**onClick**, **onChange**, etc.) ou funções passadas como **props**.

OBS: Assim como o **useMemo** é usado para **memorizar valores**, o **useCallback** **memoriza funções**, garantindo que a mesma função seja reutilizada, a menos que suas dependências mudem.

Veja abaixo sua sintaxe:

```
const callbackMemorizado = useCallback(  
  () => {  
    // Função que será memorizada  
  },  
  [dependencias] // Array de dependências  
);
```

Onde:

- **callbackMemorizado:** É a função memorizada que será recriada apenas se as dependências no array mudarem.
- **Array de dependências:** São as variáveis que, se mudarem, farão com que a função seja recriada. Se o array estiver vazio ([]), a função será criada apenas uma vez e nunca mais será recriada.

1.3.1 Por que usar useCallback?

Vimos anteriormente que o React **recria funções em toda renderização**, mesmo que elas não tenham mudado. Isso pode causar problemas de performance ou novas renderizações desnecessárias quando você passa essas funções como **props** para componentes filhos.

Usamos **useCallback** quando queremos **evitar a recriação de funções** em cada renderização, especialmente quando essas funções são passadas como **props** para componentes filhos ou usadas em hooks como **useEffect**.

1.3.2 Exemplo simples de uso do useCallback

Vamos criar um exemplo simples para mostrar como usar **useCallback**:

```
import { useState, useCallback } from "react";  
  
function App() {  
  const [contador, setContador] = useState(0);  
  
  // Função memorizada com useCallback  
  const incrementar = useCallback(() => {  
    setContador((prevContador) => prevContador + 1);  
  }, []); // Dependências vazias, a função só é criada uma vez
```

```
return (  
  <div>  
    <p>Contador: {contador}</p>  
    <button onClick={incrementar}>Incrementar</button>  
  </div>  
);  
}  
export default App;
```

Neste exemplo, o Hook **useCallback** memoriza a função **incrementar**, garantindo que a função não seja recriada em cada renderização. Isso é útil quando a função é passada como prop para componentes filhos, evitando re-renderizações desnecessárias.

1.4 React.memo

O **React.memo** é uma higher-order component (HOC) que **memoriza a renderização de um componente funcional**. Ele impede que o componente seja re-renderizado a menos que suas **props** mudem.

OBS: Um Higher Order Component (ou componente de ordem superior) é uma função que recebe um componente como argumento e retorna um novo componente. O HOC pode adicionar funcionalidades, modificar o comportamento ou mesmo alterar a renderização do componente original. HOCs são usados principalmente para implementar funcionalidades como autenticação, autorização, controle de renderização, manipulação de estado e muito mais.

1.4.1 Por que usar React.memo?

Usamos **React.memo** quando queremos **evitar que um componente inteiro seja re-renderizado**, a menos que suas **props mudem**. Isso é útil em componentes que recebem **props** que raramente mudam, permitindo otimizar a renderização.

1.4.2 Exemplo simples de uso do React.memo

Neste exemplo, o componente **Saudacao** é renderizado novamente toda vez que o **nome** é alterado, pois o nome é um dos seus **props**. No entanto, quando o endereço é alterado, o componente não é re-renderizado, já que o endereço não é passado como **prop** para **Saudacao**:

```
import { memo, useState } from "react";
// Componente pai
function App() {
  const [name, setName] = useState("");
  const [address, setAddress] = useState("");
  return (
    <>
      <label>
        Nome:
        <input
          name="nome"
          value={name}
          onChange={(e) => setName(e.target.value)}
        />
      </label>
      <br />
      <label>
        Endereço:
        <input
          name="endereco"
          value={address}
          onChange={(e) => setAddress(e.target.value)}
        />
      </label>
      <hr />
      <Saudacao name={name} />
    </>
  );
}
// Componente filho
const Saudacao = memo(({ name }) => {
  console.log("Renderizando Saudacao em", new Date().toLocaleTimeString());
  return <h3>Olá, {name}!</h3>;
});
export default App;
```

Neste exemplo:

- O componente **App** possui dois estados (**name** e **address**) que controlam os campos de entrada.
- O componente **Saudacao** é memorizado com **memo**, para que ele só seja re-renderizado quando a **prop name** mudar. Isso evita renderizações desnecessárias deste componente filho, mesmo que o estado no componente pai (**App**) mude.

Veja abaixo o resultado de sua execução após digitarmos “123” no campo de nome e “456” no campo de endereço:

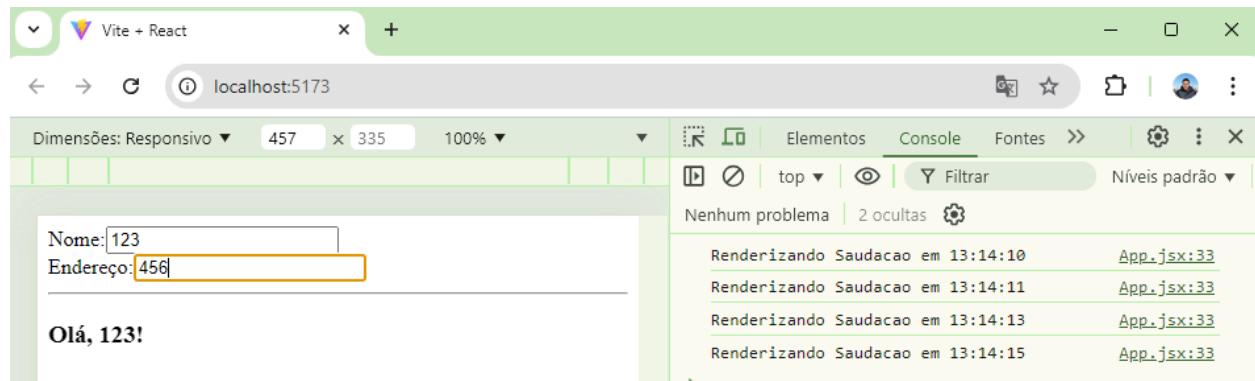


Figura 5: Exemplo de uso do React.memo

Descrição: A imagem mostra uma página web local com campos de entrada onde "Nome: 123" e "Endereço: 456" foram preenchidos. Abaixo, há uma saudação personalizada: "Olá, 123!". À direita, a aba de ferramentas do desenvolvedor exibe 4 mensagens no console do tipo "Renderizando Saudacao em"

Isso mostra que o componente **Saudacao** não é re-renderizado quando a prop **name** não muda.

1.5 Hooks Customizados

Hooks customizados são uma maneira de reutilizar lógica entre componentes React, aproveitando a capacidade dos hooks do React (como **useState**, **useEffect**, **useCallback**, etc.). Em vez de repetir a mesma lógica em diferentes componentes, você pode encapsular essa lógica em um hook customizado e reutilizá-lo em qualquer lugar da aplicação. Neste contexto, um hook customizado é basicamente uma função JavaScript que começa com "**use**" e pode combinar vários hooks internos para resolver um problema específico.

1.5.1 Vantagens de Usar Hooks Customizados

Dentre as vantagens de se usar Hooks Customizados podemos citar:

- **Reutilização de Lógica:** Permite encapsular e compartilhar lógica complexa entre vários componentes.
- **Melhor Organização:** Mantém o código limpo e fácil de manter.
- **Composição de Hooks:** Pode usar outros hooks internos (como **useState**, **useEffect**, etc.) para criar funcionalidades mais complexas.

1.5.2 Exemplo simples de uso de hook customizado

Vamos criar um hook chamado **useContador** que gerencia o estado de um **contador** e fornece funções para **incrementar**, **decrementar** e **resetar** o contador. Esse hook pode ser reutilizado em qualquer componente que precise de lógica de contador.

```
import { useState } from "react";

// Hook customizado para gerenciar um contador
function useContador(valorInicial = 0) {
  const [contador, setContador] = useState(valorInicial);
  // Função para incrementar o contador
  const incrementar = () => {
    setContador(contador + 1);
  };
  // Função para decrementar o contador
  const decrementar = () => {
    setContador(contador - 1);
  };
  // Função para resetar o contador
  const resetar = () => {
    setContador(valorInicial);
  };
  // Retorna o estado e as funções de controle
  return { contador, incrementar, decrementar, resetar };
}
```

```
// Componente que utiliza o hook customizado
function App() {
  const { contador, incrementar, decrementar, resetar } = useContador(5); //
  Começa com 5

  return (
    <div>
      <h1>Contador: {contador}</h1>
      <button onClick={incrementar}>Incrementar</button>
      <button onClick={decrementar}>Decrementar</button>
      <button onClick={resetar}>Resetar</button>
    </div>
  );
}

export default App;
```

Neste exemplo:

- Hook Customizado **useContador**:
 - **useContador** encapsula a lógica de um **contador**, inicializado com um valor opcional (**valorInicial**).
 - Ele fornece funções para **incrementar**, **decrementar**, e **resetar** o valor do contador.
 - O estado (**contador**) e as funções de controle são retornadas para o componente que usa o hook.
- Componente **App**:
 - O componente **App** utiliza o hook **useContador** para obter o valor do contador e as funções que o controlam.
 - Ao clicar nos botões, o usuário pode **incrementar**, **decrementar**, ou **resetar** o valor do **contador**.

Este exemplo mostra como você pode criar um hook customizado simples para resolver problemas comuns, como o gerenciamento de estado e funções de controle.

2. Compartilhamento de Estado Global

No React, o estado se refere aos dados que determinam como os componentes são renderizados e se comportam. Esses dados podem ser atualizados ao longo do tempo, e a atualização do estado aciona a re-renderização dos componentes. O estado pode ser categorizado em diferentes tipos com base em como e onde ele é gerenciado e utilizado.

2.1 Principais tipos de estado em React

Os principais tipos de estado em React são: Estado local e Estado global. Vamos aprofundar nossos estudos mais sobre estes tipos de estado a seguir neste curso.

2.1.1 Estado Local

O **estado local** é mantido e gerenciado dentro de um componente específico. Ele é usado para controlar e armazenar informações relevantes apenas para aquele componente (como o valor de um campo de formulário ou um contador). O estado local é gerenciado com o hook **useState** ou **useReducer** dentro de **componentes funcionais**, ou através do **this.setState** em **componentes de classe**.

Veja abaixo suas principais características:

- **Visível apenas para o componente que o define:** O estado local não pode ser acessado diretamente por outros componentes.
- **Isolado:** Cada componente tem seu próprio estado e não pode compartilhar esse estado diretamente com outros componentes. É necessário usar **props** para passar para os componentes filhos.
- **Simples:** Ideal para lógica e dados que não precisam ser compartilhados ou acessados por outros componentes da aplicação.

2.1.2 Estado Global

O **estado global** é mantido e gerenciado de forma que **vários componentes** possam acessá-lo e modificá-lo, independentemente de onde estão na árvore de componentes. O **estado global** é útil quando vários componentes precisam compartilhar os mesmos dados ou ações.

Veja abaixo suas principais características:

- **Compartilhado entre vários componentes:** O estado global pode ser acessado por diversos componentes da aplicação, sem a necessidade de passar **props** através de múltiplos níveis da árvore de componentes (prop drilling).
- **Centralizado:** O estado é gerenciado de forma centralizada, geralmente usando a [Context API](#), [Redux](#) ou outras bibliotecas de gerenciamento de estado.
- **Complexidade:** É ideal para dados e lógicas que precisam ser compartilhados em toda a aplicação (como o estado de autenticação do usuário, carrinho de compras, etc.).

OBS: Prop Drilling é um padrão no React onde as props (propriedades) são passadas de um componente para outro, muitas vezes atravessando vários níveis de componentes que não precisam diretamente dessas props, mas apenas as repassam para outros componentes mais "abaixo" na hierarquia. O problema surge quando você tem que passar dados por muitos componentes intermediários, o que pode tornar o código mais difícil de manter e entender.

2.1.3 Principais diferenças entre Estado Local e Global

Veja no quadro abaixo as principais diferenças entre Estado Local e Global:

Aspecto	Estado Local	Estado Global
Onde é gerenciado	Dentro de um componente específico.	Fora dos componentes individuais, em um contexto ou store.
Quem pode acessar	Apenas o componente que o define.	Qualquer componente que tenha acesso ao contexto ou à store global.
Como é acessado	Diretamente com useState ou setState .	Via Context API , Redux , ou outra biblioteca de estado global.
Complexidade	Simples, ideal para estados isolados.	Mais complexo, ideal para estados compartilhados entre muitos componentes.
Escopo	Escopo local do componente.	Escopo global ou compartilhado por vários componentes.
Passagem de estado	Escopo local do componente.	Não é necessário usar props, estado disponível globalmente.

2.2 Context API

A **Context API** é uma solução nativa do React que permite **compartilhar** valores ou **estados globalmente** entre componentes, sem a necessidade de passar **props** manualmente em cada nível da árvore de componentes. Ela é ideal para casos em que você precisa evitar o "**prop drilling**" — o ato de passar dados de um componente pai para vários componentes intermediários que não utilizam esses dados, apenas para que um componente mais profundo possa acessá-los.

A **Context API** é usada para **gerenciamento de estado global**, permitindo que você defina um "contexto" que pode ser acessado por qualquer componente dentro da árvore que precise desses valores.

2.2.1 Quando Usar a Context API?

A **Context API** pode ser recomendada para as seguintes situações:

- **Autenticação:** Compartilhar o estado de autenticação do usuário (se está logado ou não) e outras informações do usuário em toda a aplicação.
- **Temas:** Alternar entre temas claros e escuros em uma aplicação, sem a necessidade de passar essas informações manualmente para cada componente.
- **Idioma:** Controlar e compartilhar a escolha de idioma do usuário em toda a aplicação.
- **Configurações Globais:** Manter configurações que afetam a aplicação como um todo, como preferências de layout, notificações, etc.

2.2.2 Como Funciona a Context API?

A **Context API** funciona em três etapas principais:

1. **Criar o Contexto:** Você cria um contexto que será usado para armazenar os dados ou funções que deseja compartilhar.
2. **Provedor (Provider):** O Provider é um componente que envolve a árvore de componentes e fornece os valores (estado, funções, etc.) para os componentes que precisarem desses dados.
3. **Consumidor (Consumer):** Os componentes que precisam acessar os dados do contexto utilizam o Hook **useContext** para consumi-los.

2.2.3 Exemplo de uso da Context API

Veremos um exemplo de como usar a **Context API** para alternar entre os temas claro e escuro em nossa aplicação. O objetivo é que, ao clicar em um botão, o usuário possa alternar entre os temas, e esse tema seja aplicado globalmente na aplicação. Vejamos seus passos:

1. Criando o Contexto

O primeiro passo é criar um **contexto** com o hook **createContext()**, que é utilizado para definir quais dados serão compartilhados. O **createContext** gera um objeto de contexto que contém dois componentes principais:

- **Provider:** Responsável por fornecer os dados para os componentes que estão abaixo na árvore hierárquica de componentes.
- **Consumer** ou o **hook useContext:** Utilizado pelos componentes que desejam acessar os dados fornecidos pelo **Provider**.

Neste exemplo, crie um arquivo chamado **ThemeContext.jsx** com o seguinte código:

```
// Criando o contexto de tema (claro/escuro)
import { createContext } from 'react';

// Criando um contexto para o tema da aplicação
const ThemeContext = createContext();

export default ThemeContext;
```

Aqui estamos criando um **contexto** chamado **ThemeContext**, que armazena e compartilha o estado do **tema** (claro ou escuro) com os componentes da aplicação.

2. Definindo o Provedor (Provider)

O **Provider** é um componente especial que **envolve** os componentes que precisam acessar os dados do contexto e fornece o valor que eles poderão consumir. O **Provider** aceita uma **prop value**, que contém os dados que serão compartilhados.

Em nosso exemplo vamos considerar um arquivo chamado **ThemeProvider.jsx** com o seguinte código:

```
import { useState } from "react";
import ThemeContext from "../ThemeContext"; // Importando o contexto

// Componente Provider para gerenciar o tema
function ThemeProvider({ children }) {
  const [tema, setTema] = useState("light"); // Estado para o tema atual

  // Alterna entre tema claro e escuro
  const toggleTheme = () => {
    setTema((prevTema) => (prevTema === "light" ? "dark" : "light"));
  };

  return (
    <ThemeContext.Provider value={{ tema, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

export default ThemeProvider;
```

Aqui temos o componente **ThemeProvider**, que é fornecer o estado de **tema** e a função **toggleTheme** (que alterna entre os temas) para os componentes filhos através do **Provider**.

3. Consumindo o Contexto

Agora, vamos criar três componentes que irão **consumir o tema** e alterar a interface com base no tema atual. Em cada componente, estaremos usando [CSS inline](#) para alterar o estilo de acordo com o tema. Dependendo do valor de **tema** (claro ou escuro), as cores de fundo e do texto são ajustadas. Veja um exemplo de como pretendemos fazer a mudança de cor do tema:

- Se tema === 'light', a cor de fundo é clara (#f1f1f1), e o texto é preto.
- Se tema === 'dark', a cor de fundo é escura (#333), e o texto é branco.

Ainda, usaremos o hook **useContext** para acessar o contexto.

Primeiro, vamos criar um componente de cabeçalho (arquivo **Header.jsx**) da aplicação com o seguinte código:

```
import { useContext } from "react";
import ThemeContext from "../ThemeContext"; // Importando o contexto

function Header() {
  const { tema, toggleTheme } = useContext(ThemeContext); // Consumindo o tema e a função de alternância

  return (
    <header
      style={{
        backgroundColor: tema === "light" ? "#f1f1f1" : "#333",
        color: tema === "light" ? "#000" : "#fff",
        padding: "10px",
        textAlign: "center",
      }}
    >
      <h1>Minha Aplicação</h1>
      <button onClick={toggleTheme}>
        Alternar para tema {tema === "light" ? "escuro" : "claro"}
      </button>
    </header>
  );
}

export default Header;
```

Este código define um componente **Header** que consome o contexto **ThemeContext** para obter o estado do tema (**tema**) e a função **toggleTheme** para alternar entre os temas claro e escuro. Ele ajusta o estilo do cabeçalho com base no valor do tema: se o tema for "light", o fundo será claro e o texto será preto; se for "dark", o fundo será escuro e o texto será branco. O botão dentro do cabeçalho permite ao usuário alternar entre os dois temas dinamicamente, mudando o texto do botão de acordo com o tema atual.

Agora, vamos criar nosso componente principal da página (**Home.jsx**):

```
import { useContext } from 'react';
import ThemeContext from './ThemeContext'; // Importando o contexto

function Home() {
  const { tema } = useContext(ThemeContext);

  return (
    <main style={{
      backgroundColor: tema === 'light' ? '#fff' : '#444',
      color: tema === 'light' ? '#000' : '#fff',
      padding: '20px',
      minHeight: '100vh',
    }}>
      <p>
        Este é o conteúdo principal da página.
      </p>
    </main>
  );
}
export default Home;
```

Este código define um componente **Home** que utiliza o contexto **ThemeContext** para acessar o valor do tema atual (**tema**). Com base no valor de tema ("light" ou "dark"), o estilo do componente é ajustado. O conteúdo principal da página é exibido dentro de um elemento **<main>** com esses estilos dinâmicos aplicados.

Agora, vamos criar o componente de rodapé da nossa aplicação (**Footer.jsx**), conforme código abaixo:

```
import { useContext } from 'react';
import ThemeContext from './ThemeContext'; // Importando o contexto

function Footer() {
  const { tema } = useContext(ThemeContext);
```

```
return (  
  <footer style={{  
    backgroundColor: tema === 'light' ? '#f1f1f1' : '#333',  
    color: tema === 'light' ? '#000' : '#fff',  
    padding: '5px',  
    textAlign: 'center'  
  }}>  
    <p>&copy; 2024 Minha Aplicação</p>  
  </footer>  
);  
}  
export default Footer;
```

Este **código** define um componente **Footer** que consome o contexto **ThemeContext** para obter o valor do **tema** atual (tema). Dependendo se o tema é "light" ou "dark", o estilo do rodapé (<footer>) é ajustado. O rodapé exibe uma mensagem de copyright centralizada com estilos aplicados dinamicamente com base no tema.

4. Montando a aplicação

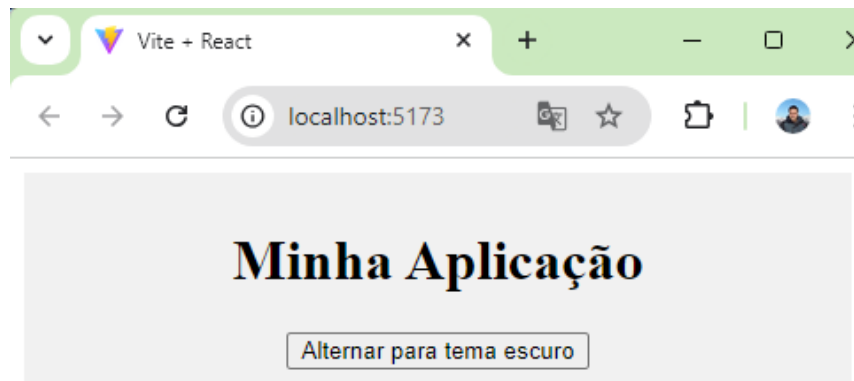
Agora vamos juntar tudo no componente **App**, onde o **ThemeProvider** envolverá os componentes filhos para que eles possam acessar o estado do tema.

```
import Header from './Header';  
import Home from './Home';  
import Footer from './Footer';  
import ThemeProvider from './ThemeProvider'; // Importando o Provider  
function App() {  
  return (  
    <ThemeProvider>  
      <Header />  
      <Home />  
      <Footer />  
    </ThemeProvider>  
  );  
}  
export default App;
```

Observe que neste exemplo:

- **Função do ThemeProvider:**
 - O **ThemeProvider** envolve os componentes filhos (**Header**, **Home**, e **Footer**). Ele é o responsável por fornecer o estado de **tema** e a função de alternância (**toggleTheme**).
 - O **Provider** usa o estado **tema** para armazenar o tema atual (light ou dark) e fornece a função **toggleTheme** para alternar entre os temas.
- **Uso do useContext:**
 - Os componentes filhos (**Header**, **Home** e **Footer**) usam o hook **useContext** para acessar o estado do tema e, no caso do **Header**, também a função **toggleTheme**.
 - Com o **useContext**, eles obtêm diretamente o valor do **tema** e ajustam o estilo (background color e color) com base no tema atual.
- **Renderização Condicional:**
 - O botão no **Header** usa o valor de **tema** para alterar seu texto e funcionalidade. Se o **tema** estiver em claro, o texto será "Alternar para tema escuro", e ao clicar, ele muda para escuro (e vice-versa).

Veja o resultado no navegador quando executamos esta aplicação:



Este é o conteúdo principal da página.

Figura 6: Exemplo de uso da Context API para definir um tema claro

Descrição: A imagem mostra uma página web. O título central da página é "Minha Aplicação", escrito em negrito. Abaixo do título, há um botão com o texto "Alternar para tema escuro", sugerindo que a página permite mudar entre temas claro e escuro. O fundo da página é cinza claro.

Em seguida, veja o resultado no navegador quando clicamos no botão “Alternar para o tema escuro”:

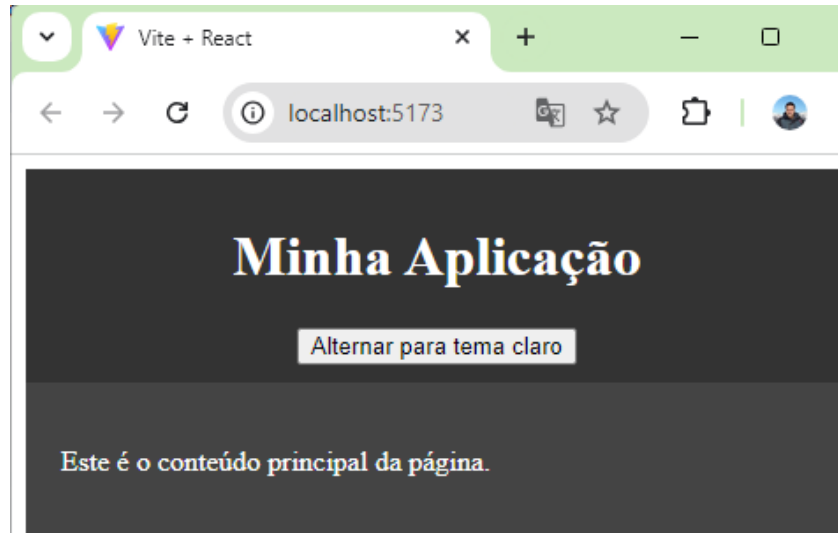


Figura 7: Exemplo de uso da Context API para definir um tema escuro

Descrição: A imagem mostra uma página web. O título central da página é "Minha Aplicação", escrito em negrito. Abaixo do título, há um botão com o texto "Alternar para tema claro", sugerindo que a página permite mudar entre temas claro e escuro. O fundo da página é preto e cinza escuro.

2.2.4 Limitações

A **Context API** pode apresentar as seguintes limitações:

- **Re-renderizações Desnecessárias:** Quando o valor do **Provider** muda, todos os componentes que consomem o contexto podem ser re-renderizados, mesmo que eles não estejam diretamente relacionados à mudança. Isso pode impactar a performance em grandes aplicações.
- **Estados Complexos:** Para gerenciamento de estado mais complexo (como múltiplas stores ou funcionalidades mais avançadas), bibliotecas como [Redux](#) ou [Zustand](#) podem ser mais adequadas.

3. Boas Práticas no Uso de Hooks em React

Os hooks são uma das funcionalidades mais poderosas do React, permitindo que você utilize estado e outras funcionalidades do React em componentes funcionais. No entanto, para manter a manutenibilidade, legibilidade e performance, é importante seguir algumas boas práticas na estruturação de lógica e no gerenciamento de efeitos colaterais ao usar hooks como **useState**, **useEffect**, **useMemo**, e **useCallback**.

A seguir estão algumas boas práticas para o uso correto de hooks, com foco em estruturação de lógica e efeitos colaterais.

3.1 Organizar a Lógica de Hooks na Ordem Correta

A ordem dos hooks é essencial para que o React possa preservá-los corretamente entre renderizações. **Nunca condicione a chamada de hooks** (ou seja, não coloque hooks dentro de blocos condicionais, loops ou funções internas). O React depende da ordem fixa dos hooks para funcionar corretamente.

Veja abaixo um exemplo incorreto de seu uso:

```
if (condition) {  
  const [state, setState] = useState(0); // Isto causará problemas  
}
```

Agora, veja abaixo um exemplo correto de seu uso:

```
const [state, setState] = useState(0); // Fora de condições  
if (condition) {  
  // Manipular o estado aqui  
}
```

3.2 Evitar useEffect sem Necessidade

Evite o uso excessivo de **useEffect** quando não for necessário. Em muitos casos, a lógica pode ser resolvida diretamente no **retorno do componente** ou em funções de eventos, ao invés de usar um efeito para controlar a renderização.

Por exemplo, em vez de usar **useEffect** para inicializar um estado com base em **props**, faça isso diretamente no **useState**.

```
function Componente({ valorInicial }) {  
  const [valor, setValor] = useState(valorInicial); // Inicializa diretamente  
  return <p>{valor}</p>;  
}
```

3.3 Usar Corretamente as Dependências no useEffect

Um erro comum é esquecer de adicionar dependências corretamente ao **useEffect**. Todas as variáveis e funções usadas dentro de **useEffect** que vêm de fora (por exemplo, **props** ou **estados**) **devem ser incluídas no array de dependências**. Isso garante que o efeito seja executado quando os valores mudarem.

Veja abaixo um exemplo incorreto de seu uso:

```
useEffect(() => {  
  console.log(valor); // 'valor' está sendo usado aqui, mas não está na lista  
  // de dependências  
}, []); // Dependências estão vazias
```

Agora, veja abaixo um exemplo correto de seu uso:

```
useEffect(() => {  
  console.log(valor); // 'valor' está corretamente incluído nas dependências  
}, [valor]); // O efeito será reexecutado quando 'valor' mudar
```

Boa prática: Procure reduzir o número de re-renderizações causadas por **useEffect**, limitando o uso de dependências complexas e garantindo que você não execute o efeito mais do que o necessário.

3.4 Dividir Efeitos Colaterais em Diferentes useEffect

Ao trabalhar com efeitos colaterais, é uma boa prática dividir diferentes efeitos que não estão relacionados entre si em múltiplos **useEffect**. Isso torna o código mais limpo, previsível e facilita a manutenção.

Veja abaixo um exemplo incorreto de seu uso:

```
useEffect(() => {  
  fetchData();  
  document.title = `Contador: ${contador}`;  
}, [contador]);
```

Agora, veja abaixo um exemplo correto de seu uso:

```
useEffect(() => {  
  fetchData();  
}, []); // Efeito para buscar dados, sem relação com o contador  
  
useEffect(() => {  
  document.title = `Contador: ${contador}`;  
}, [contador]); // Efeito para alterar o título com base no contador
```

3.5 Uso de `useMemo` e `useCallback` para Otimizações

Os hooks **`useMemo`** e **`useCallback`** são úteis para memorizar valores computados e funções que não precisam ser recriados a cada renderização. Use **`useMemo`** para evitar recalcular valores caros, e **`useCallback`** para evitar recriar funções em cada renderização desnecessariamente.

- **`useMemo`**: Memoriza o valor de uma função de computação para evitar re-executá-la a cada renderização.
- **`useCallback`**: Memoriza a função para que ela não seja recriada em toda renderização.

3.6 Encapsular Lógica Reutilizável em Hooks Customizados

Se você notar que está repetindo muita lógica que envolve hooks em diferentes partes da aplicação, uma boa prática é encapsular essa lógica em **hooks customizados**. Isso torna o código mais limpo e reutilizável.

Veja abaixo um exemplo correto de uso de Hooks Customizados:

```
function useContador(valorInicial = 0) {
  const [contador, setContador] = useState(valorInicial);

  const incrementar = () => setContador(contador + 1);
  const decrementar = () => setContador(contador - 1);

  return { contador, incrementar, decrementar };
}

// Utilizando o hook customizado
function ContadorComponent() {
  const { contador, incrementar, decrementar } = useContador(10);
  return (
    <div>
      <p>Contador: {contador}</p>
      <button onClick={incrementar}>+</button>
      <button onClick={decrementar}>-</button>
    </div>
  );
}
```

O hook customizado **useContador** encapsula a lógica de gerenciamento de um contador (com funções de incrementar e decrementar) e facilita sua reutilização em diferentes componentes. As vantagens de usar um hook customizado neste exemplo incluem:

- **Reutilização:** A lógica do contador pode ser usada em vários componentes sem repetir o código.
- **Organização:** A lógica de estado é separada da interface, tornando o componente mais limpo e fácil de entender.
- **Facilidade de manutenção:** Qualquer alteração na lógica do contador pode ser feita diretamente no hook, refletindo em todos os componentes que o utilizam.

Referências

O material desenvolvido para este capítulo baseou-se nas seguintes obras:

- DEVLIN BASILAN DULDULAO, D. B.; CABAGNOT, R. J. L. Practical Enterprise React: Become an Effective React Developer in Your Team. USA: APRESS, 2021.
- ELROM, E. React and Libraries: Your Complete Guide to the React Ecosystem. USA: APRESS, 2021.
- MDN WEB DOCS. Getting started with React. Disponível em: https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side_JavaScript_frameworks/React_getting_started. Acessado em 21 jun. 2024.
- REACT DOCS. Disponível em <https://pt-br.react.dev/>. Acessado em 15 jun. 2024.
- ROLDÁN, C. S. React 17: Design Patterns and Best Practices. UK: Packt Publishing, 2021.
- SCOTT, A. D. JavaScript Everywhere. USA: O'Reilly Media, 2020.
- UZAYR, S. B.; CLOUD, N.; AMBLER, T. JavaScript Frameworks for Modern Web Development: The Essential Frameworks, Libraries, and Tools to Learn Right Now. USA: APRESS, 2019.
- W3SCHOOLS. React Tutorial. Disponível em: <https://www.w3schools.com/react>. Acessado em 17 jun. 2024.